

CPT102 Data Structures:

Lecture 5: Searching, sorting

Thomas Selig

XJTLU

March 30, 2026

- Week 3 practice exercises solutions on Learning Mall.
- This week's Office Hour availability:
 - Monday 14:00-15:00;
 - Tuesday 16:00-17:00;
 - Friday 13:00-14:00.

Other times by appointment

- CW (Phase 1) is here!
 - Groups have been set; consistent with Lab groups.
 - Task sheet in on Learning Mall.
 - Short introduction.

Contents of today's lecture

More algorithms: searching and sorting

- Searching: linear search, binary search
- What is sorting?
- A few classical sorting algorithms:
 - Bubble sort;
 - Selection sort;
 - Insertion sort.
- As usual: complexity analysis.

Lecture Learning Outcomes

By the end of today's lecture, you should be able to:

- A. Define the concept of an order relation, illustrating with standard examples.
- B. Implement linear and binary search algorithms in pseudocode, using both iterative and recursive techniques.
- C. Explain basic sorting algorithms based on a decrease-and-conquer approach, such as bubble sort, selection sort, or insertion sort, and implement them in pseudocode.
- D. Analyse the complexity of searching and sorting algorithms.

Reminder: linear search

Recall from previous weeks:

Algorithm 1: iterative linear search

Procedure ITERATIVELINEARSEARCH(*arr*, *val*)

foreach *item* **in** *arr* **do**

if *item* = *val* **then**
 return true;

return false;

- Simple iterative version.
- **Linear** time complexity $O(n)$, **constant** space complexity $O(1)$.

Recursive linear search

Last week, we also saw:

Algorithm 2: recursive linear search

Procedure LINEARSEARCHHELPER(*arr*, *val*, *startIndex*)

Output: **true** if *arr* contains *val* at an index $\geq$ *startIndex*, **false** otherwise

if *startIndex* = *arr.length* **then**

return false; ▷ base case

if *arr*[*startIndex*] = *val* **then**

return true;

return LINEARSEARCHHELPER(*arr*, *val*, *startIndex* + 1);

Procedure RECURSIVELINEARSEARCH(*arr*, *val*)

return LINEARSEARCHHELPER(*arr*, *val*, 0);

Linear time and space complexities.

Question

Is there a sorting algorithm that terminates faster than in linear time?

- For general inputs, yes!
 - Need to check each element one-by-one.
- What about “non-general” inputs, i.e. inputs with additional structure or constraints?
 - Consider **sorted** inputs!

Searching sorted arrays

- Suppose that we have an array `arr` of numbers, which are indexed in increasing order.
 - That is: $\text{arr}[0] < \text{arr}[1] < \dots < \text{arr}[n - 1]$
- We are searching for a value `val`.
- Fix an index m .
 - If $\text{arr}[m] = \text{val}$, we're done.
 - If $\text{arr}[m] < \text{val}$, then by the sorted property `val` can **only** be found in $\text{arr}[m + 1 : n]$.
 - Similarly, if $\text{arr}[m] > \text{val}$, then `val` can only be found in $\text{arr}[0 : m]$.
- If we take $m = \frac{n}{2}$, this allows us to “throw away” half of the array.
 - This is the idea of binary search, and on divide-and-conquer algorithms in general.

N.B.: In the above, the notation $\text{arr}[a : b]$ is short-hand for the sub-array given by $[\text{arr}[a], \text{arr}[a + 1], \dots, \text{arr}[b - 1]]$

The binary search algorithm: recursive version

As in RECURSIVELINEARSEARCH, to avoid extracting sub-arrays, we use a helper algorithm.

- Helper takes two additional parameters: `startIndex` and `endIndex`.

Procedure BINARYSEARCHHELPER(`arr`, `val`, `startIndex`, `endIndex`)

Output: `true` if there exists an index i with $\text{startIndex} \leq i < \text{endIndex}$ such that $\text{arr}[i] = \text{val}$, `false` otherwise

if `startIndex = endIndex` **then**

return `false`;

`midIndex` $\leftarrow$ (`startIndex` + `endIndex`) / 2; ▷ integer division, like in Java

if `arr[midIndex] = val` **then**

return `true`;

else if `arr[midIndex] < val` **then**

return BINARYSEARCHHELPER(`arr`, `val`, `midIndex` + 1, `endIndex`);

else

return BINARYSEARCHHELPER(`arr`, `val`, `startIndex`, `midIndex`);

Procedure BINARYSEARCH(`arr`, `val`)

return BINARYSEARCHHELPER(`arr`, `val`, 0, `arr.length`);

Complexity analysis: BINARYSEARCH (1/2)

- Complexity of BINARYSEARCH is just the complexity of BINARYSEARCHHELPER.
- The base case of BINARYSEARCHHELPER is reached when we have $\text{startIndex} = \text{endIndex}$.
 - Input size is therefore $n := \text{endIndex} - \text{startIndex}$.
- Key observations:
 - Each recursive step executes at most one recursive call.
 - On any recursive call, the size of the input is at most $n/2$.
 - All other operations are just constant time.
- This gives:
$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

Complexity analysis: BINARYSEARCH (2/2)

- We have: $T(n) = T\left(\frac{n}{2}\right) + O(1)$
- What is $T(n)$?
- Since each recursive call adds constant time, $T(n)$ is essentially the number of recursive calls.
 - In each recursive call, input size is divided by 2.
 - After one recursive call, input size is $n/2$.
 - After two recursive calls, input size is $(n/2)/2 = n/4$.
 - After k recursive calls, input size is $n/2^k$.
- The total number of recursive calls is therefore the value of k such that $n/2^k \approx 1$.
 - This is in fact just $k = \log_2(n)$
- Final analysis: $T(n) = O(\log(n))$
- Space complexity is the size of the call stack, which again is the number of recursive calls, so $M(n) = O(\log(n))$

The binary search algorithm: iterative version

Recall that any recursive algorithm has an iterative transcription.

- Generally, iterative version is (schematically):
 while (base case not reached)
 repeat recursive step

Procedure ITERATIVEBINARYSEARCH(*arr*, *val*)

startIndex $\leftarrow$ 0, *endIndex* $\leftarrow$ *arr*.length;

while *startIndex* < *endIndex* **do**

midIndex $\leftarrow$ (*startIndex* + *endIndex*) / 2;

if *arr*[*midIndex*] = *val* **then**

return true;

else if *arr*[*midIndex*] < *val* **then**

startIndex $\leftarrow$ *midIndex* + 1;

else

endIndex $\leftarrow$ *midIndex*;

return false;

Sorting: what does it mean?

Question

Why did Slide 6 specify that `arr` is an array of **numbers**?

- We require our array elements to be **sorted**, i.e. indexed in **increasing order**.
- Our algorithm requires **comparisons** between pairs of values.

This requires an **order relation** on the types of element in the array.

Order relation: definition

Given a set A , a **binary relation** on A is a mapping:

$\mathcal{R} : A \times A \rightarrow \{\mathbf{true}, \mathbf{false}\}$. We write $a_1 \mathcal{R} a_2$ when $\mathcal{R}(a_1, a_2) = \mathbf{true}$.

Definition

A (total) **order relation** on a set A is a binary relation $<$ satisfying the following two conditions:

- *Trichotomy*: For any pair $(a, b) \in A \times A$, exactly one of $a < b$, $a = b$, $b < a$ holds.
- *Transitivity*: For any triple (a, b, c) , if $a < b$ and $b < c$, then we have $a < c$.

We say that $(A, <)$ is an *ordered set*.

Order relation: examples

What can we order?

- Numbers, at least integers and reals;
- Complex numbers is more ...complex;
 - Can actually show that there is no order relation on the set of complex numbers which is compatible with multiplication;
- Letters can be ordered in alphabetical order;
 - Do we distinguish upper- and lower-case?
- Others?

In Java, order relations are defined by implementing the `Comparator` and `Comparable` interfaces (see tutorial).

Order relation: activity

- Our university stores a collection of students (e.g., in e-Bridge).
- Want to maintain this in a sorted array/list.
- What criteria should we use to sort students by?
 - Single criterion (e.g., ID number), or combination of multiple criteria?

Share your thoughts in the Etherpad activity here.

Lexicographical order: definition

Suppose we have ordered sets $(A_1, <_1), (A_2, <_2), \dots$, and we want to order sequences $(a_1, a_2, \dots) \in A_1 \times A_2 \times \dots$

- Classic example: we know how to order *letters*, how do we order *words* (sequences of letters)?

Definition

The **lexicographical order** $<_{\text{lex}}$ on $A_1 \times A_2 \times \dots$ is defined as follows. For two sequences $a := (a_1, a_2, \dots)$, $a' := (a'_1, a'_2, \dots)$, we say that $a <_{\text{lex}} a'$ if there exists $i \geq 1$ such that $a_i <_i a'_i$ and $a_j <_j a'_j$ for all $j < i$.

How does the lexicographical order work?

- Compare a_1 and a'_1 . If $a_1 < a'_1$ then $a < a'$; if $a'_1 < a_1$ then $a' < a$.
- If $a_1 = a'_1$, compare a_2 and a'_2 as above, then repeat.

Lexicographical order: examples

Intuition

Lexicographical orders provide a way of combining ordered sets.

- Dictionary order is just lexicographical order on letters.
 - Needs convention when one word is a prefix of the other, e.g. “learn” and “learner”.
- Useful for ordering in databases, e.g. students at XJTLU. For example, we can order by (`last_name`, `first_name`, `year_of_study`, `id_number`)
 - Note that `last_name` and `first_name` orders are already lexicographical orders.

Sorting algorithms

Quick recap:

- What sorting means;
- Why sorting matters.

Question

How to sort?

- This week, will see three basic sorting algorithms: bubble sort, insertion sort, and selection sort.
- Next week: divide-and-conquer techniques, including merge sort and quick sort, for faster sorting.
 - Also two techniques on specific inputs: bucket sort and radix sort.

Definition

A **general sorting algorithm** is an algorithm that sorts an input array using only pairwise *comparisons* of elements and positional *swaps*.

Our general framework therefore assumes:

- Input: an array whose elements are members of a given ordered set $(A, <)$; for example, an array of numbers.
- Two elementary operations:
 - $\text{LESS}(a, b)$, which returns **true** if $a < b$, and **false** otherwise;
 - $\text{SWAP}(\text{arr}, i, j)$, which swaps the elements of `arr` at indices i and j , leaving other elements unchanged.

Both operations are assumed to run in constant $O(1)$ time, irrespective of their inputs.

Bubble sort: overview

Our first sorting algorithm: **bubble sort**.

- Algorithm is a series of *passes* through the array.
- On each pass, compare pairs of *consecutive* elements, swap them if they are in the wrong order.
- Repeat until the array is sorted.
- Name comes from the fact that the largest elements “bubble” towards the top (end) of the array.

Pseudocode for one pass:

```
for  $i \leftarrow 0$  to  $j$  do  
    if LESS( $arr[i+1]$ ,  $arr[i]$ ) then  
        SWAP( $arr$ ,  $i$ ,  $i+1$ );
```

Bubble sort: example

Input array: [6, 10, 4, 2, 8]

① First pass:

- ① Start from $i = 0$: we have $\text{arr}[0] = 4 < 10 = \text{arr}[1]$. Elements are in the right order, no need to swap.
- ② Next, $i = 1$: we have $\text{arr}[1] = 10 > 6 = \text{arr}[2]$. Elements need to be swapped, so $\text{arr} \rightarrow [6, 4, 10, 2, 8]$.
- ③ Next, $i = 2$: we have $\text{arr}[2] = 10 > 2 = \text{arr}[3]$. Elements need to be swapped, so $\text{arr} \rightarrow [6, 4, 2, 10, 8]$.
- ④ Next, $i = 3$: we have $\text{arr}[3] = 10 > 8 = \text{arr}[4]$. Elements need to be swapped, so $\text{arr} \rightarrow [6, 4, 2, 8, 10]$.

Array is not sorted, so need another pass.

- ② Second pass: swap 6 and 4, then 6 and 2; $\text{arr} \rightarrow [4, 2, 6, 8, 10]$.
Array not sorted, so need third pass.
- ③ Third pass: swap 4 and 2, no other swaps; $\text{arr} \rightarrow [2, 4, 6, 8, 10]$.
Array now sorted, so we can stop.

Bubble sort: implementation

- Key observation: after first pass, the largest element of the array will be moved to its correct position at the end of the array.
- As such, next pass can stop one position earlier, and so on.

This gives us the following pseudocode implementation:

Algorithm 3: Bubble sort

Procedure BUBBLESORT(arr)

$n \leftarrow \text{arr.length}$; swapsNeeded $\leftarrow$ **false**; $k \leftarrow n - 1$;

▷ k is the index to which the largest remaining element “bubbles up” to on each pass

while $k > 0$ **do**

for $i \leftarrow 0$ **to** $k - 1$ **do**

if LESS(arr[i + 1], arr[i]) **then**

 SWAP(arr, i, i + 1);

 swapsNeeded $\leftarrow$ **true**;

if !swapsNeeded **then**

return ;

 ▷ if no swaps needed on a whole pass, the array is sorted

Bubble sort: complexity analysis

Time complexity

- Sufficient to count comparisons, since $0 \leq \#swaps \leq \#comparisons$.
- Inner loop always has k comparisons.
- In worst case, where the original array is in reverse order, we can see that swaps are needed on each pass, so all passes are necessary.

- Worst-case number of comparisons is therefore $\sum_{k=1}^{n-1} k = \frac{n(n-1)}{2}$.

- This gives: $T_{\text{worst}}(n) = O(n^2)$

- Average case (complicated) also gives $T_{\text{avg}}(n) = O(n^2)$

- Best case: array is already sorted, first pass detects that no swaps are needed, giving $T_{\text{best}}(n) = O(n)$

Space complexity

- No additional memory required, since algorithm is **in-place**, so space complexity is $M(n) = O(1)$

Insertion sort: overview

Our second sorting algorithm: **insertion sort**

- Arguably the most natural sorting algorithm: how you would sort e.g. a stack of exam papers.
- High-level overview:
 - Start with a stack of elements, and an empty list.
 - While the stack is non-empty, repeat:
 - Pop the stack;
 - Insert the element popped into its correct position in the list.
- In practice, implementing the above requires creating a new array, so is wasteful in memory.
- Instead, we implement an in-place version where the array is split into a sorted portion $\text{arr}[0 : i]$ and an unsorted portion $\text{arr}[i : n]$.
 - For i ranging from 1 to $n - 1$, we insert $\text{arr}[i]$ (first unsorted element) into its correct position in the sorted portion.

Insertion sort: implementation

Algorithm 4: Insertion sort

Procedure INSERTIONSORT(*arr*)

for $i \leftarrow 1$ **to** *arr.length* - 1 **do**

$j \leftarrow i$;

▷ insertion index

while $j > 0$ **and** LESS(*arr*[*j*], *arr*[*j* - 1]) **do**

 SWAP(*j*, *j* - 1);

$j \leftarrow j - 1$;

- Standard implementation uses a shifting mechanism (see Week 3 INSERT(*list*, *item*) procedure) rather than swaps.
 - Faster, as a swap is at least two assignments (three in most languages).
- But our chosen model allows only swaps and comparisons, so we present a swap-based version.

Insertion sort: complexity analysis

Time complexity

- If we denote $R(i)$ for the number of repetitions of the inner loop, then the complexity is $\sim \sum_{i=1}^{n-1} R(i)$.
- Worst-case: array is in reverse order, each successive element bubbles all the way to the front of the array, giving $R(i) = i$.
Complexity is again $T(n) = O(n^2)$ (quadratic).
- Average-case: on average, insertions are in the middle of the sorted portion, so $R(i) \approx \frac{i}{2}$. Improves only by constant, still quadratic $O(n^2)$ complexity.
- Best-case: if the array is sorted, no swaps are needed, i.e. $R(i) = 1$, so that $T(n) = O(n)$ (linear).

Space complexity

- In-place sorting: $M(n) = O(1)$.

Our third and final (for today) sorting algorithm: **selection sort**

- Idea is simple: remove minimal elements from the list one-by-one, appending them to the end of the sorted list.
- As with insertion sort, we implement an in-place version where the array is split into a sorted portion `arr[0 : i]` and an unsorted portion `arr[i : n]`.
 - For i ranging from 0 to $n - 2$, find the minimum element in `arr[i : n]` (minimum remaining unsorted element) and place it at index i .

Selection sort: implementation

Algorithm 5: Selection sort

```
 $n \leftarrow \text{arr.length};$   
for  $i \leftarrow 0$  to  $n - 2$  do  
     $\text{minIndex} \leftarrow i; \text{currentMin} \leftarrow \text{arr}[i];$   
    for  $j \leftarrow i + 1$  to  $n - 1$  do  
        if  $\text{LESS}(\text{arr}[j], \text{currentMin})$  then  
             $\text{minIndex} \leftarrow j; \text{currentMin} \leftarrow \text{arr}[j];$   
     $\text{SWAP}(i, \text{minIndex});$ 
```

Selection sort: complexity analysis

- Loops are fixed size for-loops, so the time complexity will be the same in best, worst and average cases.
- Outer loop has $\approx n$ iterations, inner loop has $\approx n - i$ iterations, giving

$$T(n) = O(n^2) \text{ (quadratic).}$$

- However, the complexity is borne by the *comparisons*, since the number of *swaps* is $n - 1 = O(n)$.
 - If swaps are much more costly than comparisons, this makes selection sort a better choice than other algorithms seen so far.
- Space complexity is still $M(n) = O(1)$ (constant, since in-place).

Towards faster sorting

- So far, our three algorithms split the array into a sorted part and an unsorted part, with each “step” moving a single element from the unsorted part into the sorted part.
 - That is, each step decreases the size of the unsorted part by one.
- We can think of this as a recurrence relation: $T(n) = T(n-1) + f(n)$, where $f(n)$ is the additional cost of the step.
 - Bubble sort: step bubbles the largest (remaining) value to the end of the array.
 - Insertion sort: a bit like a reverse bubble sort.
 - Selection sort: step finds minimal remaining element, swaps it with the first unsorted element.
- All of these steps have additional **linear** complexity, so in total we get $T(n) = T(n-1) + O(n)$, hence quadratic complexity.
- Next week, we will see several faster algorithms based on the so-called **divide-and-conquer** approach, which splits the array into two parts of size $n/2$ at each step.

- Concept of an ordered set, allowing for sorting data structures.
- Searching: reminder of linear search, binary search for sorted arrays, recursive and iterative versions.
- Sorting: bubble sort, insertion sort, selection sort.
 - In-place implementations to optimise memory usage.
- Time complexity analyses of all algorithms.